

PROJECT REPORT: CRICKET ANALYTICS & MANAGEMENT SYSTEM

Submitted By: Saad Nanawala

Institution: Government Engineering College (GEC), Gandhinagar

Domain: Full-Stack Web Development & Database Management

1. Introduction

The Cricket Analytics and Management System is a full-stack web application developed to handle sports player rosters and performance metrics. The primary objective of this project is to implement a robust CRUD (Create, Read, Update, Delete) architecture, connecting a relational database with a Python backend and a modern asynchronous frontend.

2. System Architecture & Technology Stack

The application is structured into a clean 3-tier architecture:

Presentation Layer (Frontend): Pure HTML5, CSS3, and native JavaScript using the asynchronous `fetch()` API for seamless, page-reload-free interactions.

Application Layer (Backend): Python utilizing the Flask micro-framework to handle routing, business logic, and RESTful API endpoints.

Data Persistence Layer (Database): MySQL 8.0 relational database managed via `mysql-connector-python`.

3. Database Schema Design

The backend communicates with a MySQL database named `cricket_db`. Below is the schema structure used for data storage:

SQL

```
CREATE DATABASE IF NOT EXISTS cricket_db;  
USE cricket_db;
```

```
CREATE TABLE players (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    role VARCHAR(50) NOT NULL,  
    matches INT DEFAULT 0,  
    runs INT DEFAULT 0,  
    wickets INT DEFAULT 0  
);
```

4. Backend REST API Endpoints (app.py)

The Flask application exposes standard JSON-based API routes to process client requests:

GET /api/players: Connects to MySQL, executes a SELECT query, and returns all player records as a JSON array.

POST /api/players: Accepts incoming JSON payloads and executes parameterized SQL INSERT statements to secure against SQL injection vulnerabilities.

PUT /api/players/<id>: Modifies existing player records based on their unique primary key ID.

DELETE /api/players/<id>: Removes a specific player record from the database table.

5. Frontend Multi-Page Interface

The user interface consists of four distinct pages styled with a uniform modern design system:

Dashboard (index.html): Serves as the landing page displaying system status and technical stack metrics.

Player Roster (roster.html): The core CRUD management portal featuring an input form for adding/editing records and a dynamic table displaying live data.

Analytics (stats.html): Aggregates real-time statistics from the database (such as total squad members, total runs, and total wickets).

About (about.html): Documents the system architecture and engineering methodologies used.

6. Conclusion

This project successfully demonstrates the practical implementation of a full-stack web application. By decoupling the backend REST API from the frontend interface and securing database transactions, the system provides a scalable foundation for sports data management and future machine learning performance predictions. interface and securing database transactions, the system provides a scalable foundation for sports data management and future machine learning performance predictio